

CONTROL DE ACCESO A LA RED (RADIUS / AAA)

RAFAEL GUAJARDO SALVO
TÉCNICO EN CIBERSEGURIDAD

ÍNDICE:

Introducción	3
RADIUS.....	4
A.A.A (authentication – authorization – Accounting)	8
Laboratorio:	9
Vinculación de Ubuntu al dominio YORHA CENTER	12
Vincular RADIUS con Active Directory	16
Conclusión:	23

Introducción

Este trabajo presenta la implementación de un entorno de autenticación centralizada utilizando FreeRADIUS integrado con Active Directory. A lo largo del laboratorio se configuró un servidor Linux capaz de comunicarse con el dominio, validar credenciales mediante winbind y utilizar ntlm_auth como puente hacia AD. El objetivo fue construir un flujo realista de autenticación empresarial, comprendiendo tanto la integración técnica como las diferencias entre métodos como PAP y MSCHAP, así como las limitaciones de las herramientas de prueba utilizadas.

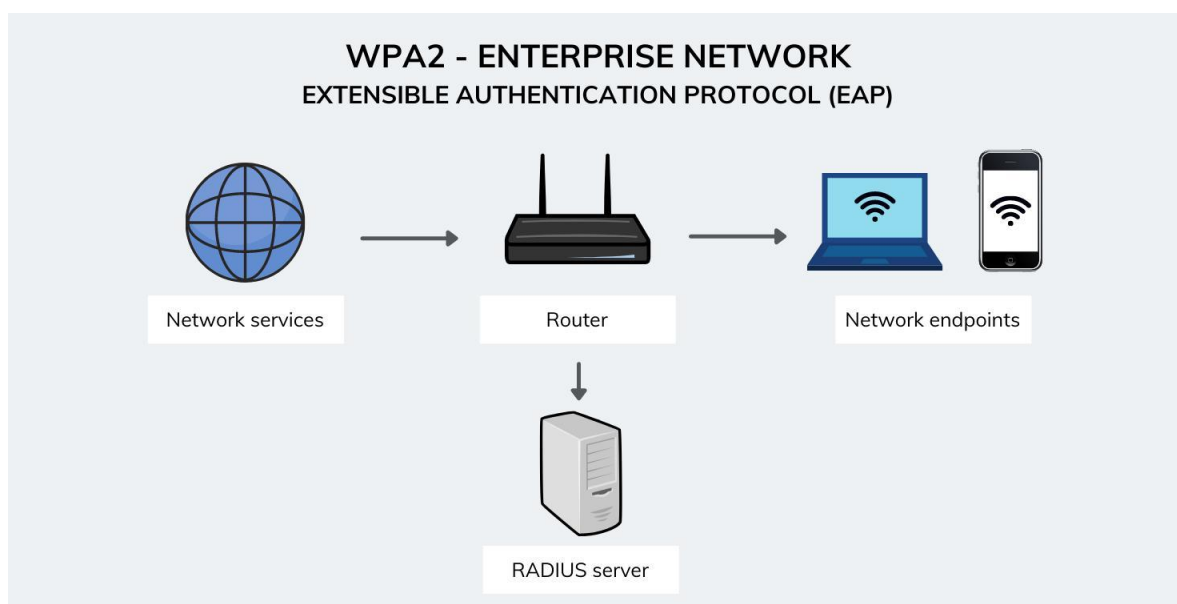
RADIUS

RADIUS (Remote Authentication Dial-In User Service) es un protocolo de red que permite centralizar la autenticación, autorización y registro de usuarios (AAA) en una infraestructura. Su función principal es que dispositivos como routers, firewalls o puntos de acceso no validen directamente las credenciales, sino que deleguen esa tarea a un servidor RADIUS.

Cuando un usuario intenta acceder a la red, el dispositivo intermediario envía la solicitud al servidor RADIUS, el cual verifica la identidad y responde indicando si el acceso está permitido o no. De esta forma, se logra un control unificado, más seguro y administrable, ya que todas las decisiones de acceso se gestionan desde un solo punto.

En términos prácticos, RADIUS permite que una organización controle quién puede conectarse a sus recursos, qué permisos tiene y registre su actividad, siendo ampliamente utilizado en entornos como redes WiFi-empresariales, VPN y acceso a equipos de red.

En una empresa, el acceso a la red WiFi corporativa está controlado mediante un servidor RADIUS. Cuando un empleado intenta conectarse, introduce su usuario y contraseña en su dispositivo. Sin embargo, el punto de acceso WiFi no valida directamente estas credenciales, sino que las envía al servidor RADIUS para su verificación.



El servidor RADIUS consulta una base de datos centralizada, como Active Directory, para comprobar si el usuario existe y si la contraseña es correcta. Si la validación es exitosa, responde con un “Access-Accept”, permitiendo el acceso a la red; en caso contrario, responde con un “Access-Reject”, denegando la conexión.

Este modelo permite a la empresa gestionar el acceso de forma centralizada, ya que todas las decisiones de autenticación se realizan desde un único punto. Por ejemplo, si un empleado deja la organización, basta con desactivar su cuenta en el servidor para que automáticamente pierda acceso al WiFi, la VPN y otros recursos de red. Además, el sistema puede registrar la actividad de los usuarios, facilitando el control y la auditoría de accesos.

Acceso a VPN corporativa

En muchas organizaciones, el acceso remoto a la red interna se realiza mediante una VPN integrada con RADIUS. Cuando un usuario intenta conectarse desde fuera de la empresa, el firewall o concentrador VPN envía sus credenciales al servidor RADIUS. Este, valida la identidad y determina si el usuario tiene permiso para conectarse. De esta forma, se controla de manera centralizada quién puede acceder a la red interna desde internet.

Acceso a switches (administración de red)

En entornos empresariales, los administradores de red no se autentican localmente en cada switch o router. En su lugar, estos dispositivos están configurados para usar RADIUS. Cuando un administrador intenta acceder por SSH o consola, el equipo envía la solicitud al servidor RADIUS, que valida las credenciales y puede incluso asignar distintos niveles de privilegio según el usuario. Esto evita tener múltiples cuentas locales distribuidas en cada equipo.

Control de acceso por cable (802.1X)

En redes corporativas avanzadas, no basta con conectarse físicamente a un puerto de red. Mediante 802.1X, el switch solicita autenticación al usuario o dispositivo conectado. Esta solicitud se valida a través de RADIUS. Solo si el servidor autoriza el acceso, el puerto se habilita. Esto previene accesos no autorizados dentro de la red interna.

Redes WiFi para invitados

Muchas empresas ofrecen WiFi para visitantes separado de la red interna. Con RADIUS, se puede gestionar el acceso de invitados de forma controlada. Por ejemplo, se generan credenciales temporales que el servidor valida. Además, se pueden aplicar restricciones, como acceso solo a internet, limitación de ancho de banda o expiración automática del acceso.

Control de dispositivos y políticas de red

RADIUS también se utiliza para asignar políticas dinámicas según el usuario o dispositivo. Por ejemplo, al autenticarse, un usuario puede ser asignado automáticamente a una VLAN específica. Un empleado puede acceder a recursos internos, mientras que un dispositivo desconocido puede ser aislado en una red restringida. Esto permite segmentar la red sin necesidad de configuraciones manuales en cada equipo.

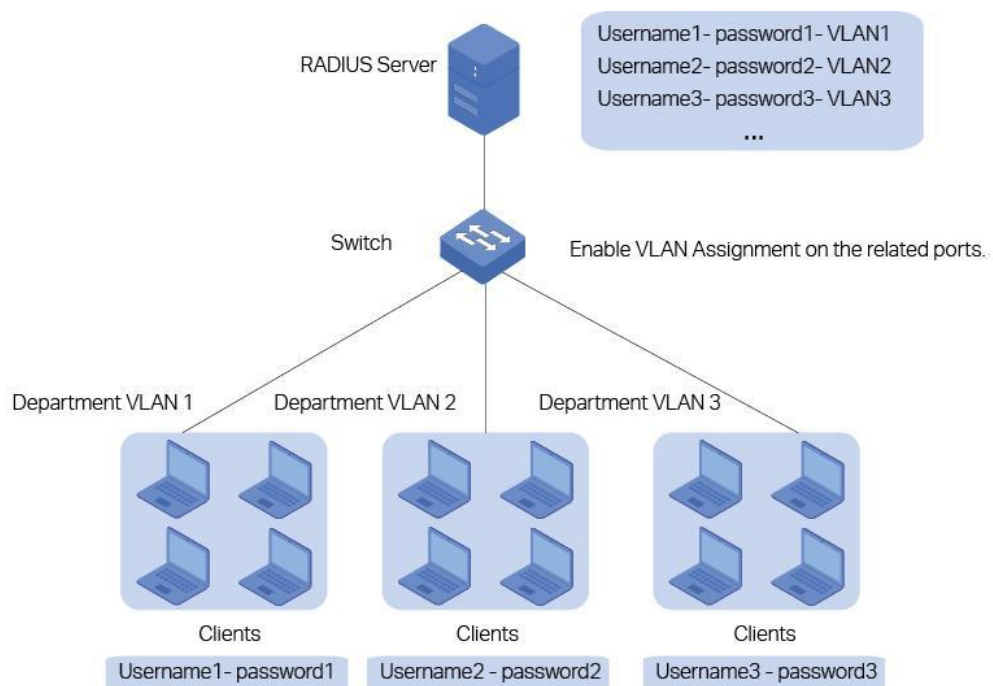
Conclusión de RADIUS:

- centralizar la autenticación y el control de acceso
- definir permisos según el usuario
- registrar la actividad para auditoría

RADIUS y la VLAN

RADIUS y la segmentación de red no cumplen la misma función, pero trabajan juntos de forma directa para controlar el acceso dentro de una infraestructura. RADIUS se encarga de autenticar al usuario y definir qué nivel de acceso debería tener, mientras que la segmentación, a través de mecanismos como VLAN o ACL, se encarga de aplicar ese acceso dentro de la red.

En la práctica, cuando un usuario intenta conectarse, el dispositivo de red envía sus credenciales al servidor RADIUS. Este, valida la identidad y, además de permitir o denegar el acceso, puede indicar a qué segmento de red debe pertenecer ese usuario. Con esa información, el switch o firewall lo ubica en una VLAN específica y le aplica las políticas correspondientes.



Ejemplo real:

- Te conectas a la red
- RADIUS valida tu usuario: Según quién eres, responde algo como: “usuario válido + asignar VLAN 10”
- El switch/firewall hace el resto: te mete en esa VLAN
- aplica reglas

A.A.A (authentication – authorization – Accounting)

AAA es un modelo de control de acceso que utiliza RADIUS para gestionar de forma centralizada cómo los usuarios ingresan y operan dentro de una red. Está compuesto por tres etapas que trabajan en conjunto. Primero, la **autenticación (Authentication)** verifica la identidad del usuario, es decir, confirma que realmente es quien dice ser mediante credenciales como usuario y contraseña. Una vez validada la identidad, entra la **autorización (Authorization)**, que determina qué nivel de acceso tendrá dentro de la red, por ejemplo, a qué recursos puede acceder o en qué segmento de red será ubicado según su perfil o grupo. Finalmente, la **contabilización (Accounting)** registra toda la actividad relacionada con ese acceso, como horarios de conexión, duración de la sesión o recursos utilizados, lo que permite llevar control, auditoría y trazabilidad. En conjunto, AAA permite no solo decidir quién puede entrar, sino también controlar qué puede hacer y dejar evidencia de su comportamiento dentro de la red.

RADIUS implementa este modelo AAA como una forma de gestión centralizada del acceso.

Flujo completo

1. El usuario intenta conectarse
2. El dispositivo de red envía la solicitud al servidor RADIUS
3. RADIUS autentica al usuario
4. Si la identidad es válida, RADIUS autoriza el nivel de acceso
5. Durante o después de la sesión, RADIUS registra la actividad

Laboratorio:

Primero verificar alcance y topología de la red, confirmando las IP de nuestros dispositivos:

Ubuntu Linux: 192.168.1.86

Windows server 2025 192.168.1.89

Windows 10: 192.168.1.88

Confirmamos si hay conexión entre ellas generando Ping

Desde Windows 10:

```
C:\Windows\system32>ping 192.168.1.86

Haciendo ping a 192.168.1.86 con 32 bytes de datos:
Respuesta desde 192.168.1.86: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.1.86: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.1.86: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.1.86: bytes=32 tiempo<1m TTL=64

Estadísticas de ping para 192.168.1.86:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
              (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 1ms, Media = 0ms

C:\Windows\system32>ping 192.168.1.89

Haciendo ping a 192.168.1.89 con 32 bytes de datos:
Respuesta desde 192.168.1.89: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.1.89: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.1.89: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.1.89: bytes=32 tiempo<1m TTL=128

Estadísticas de ping para 192.168.1.89:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
              (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 0ms, Media = 0ms

C:\Windows\system32>
```

Una vez confirmada la conexión entre los dispositivos, instalarás free radius en Ubuntu que se utilizará como servidor:

sudo apt update

sudo apt install freeradius freeradius-utils -y

```
*****
Configurando freeradius (3.2.5+dfsg-3~ubuntu24.04.3) ...
update-rc.d: warning: start and stop actions are no longer supported; falling back to defaults
Created symlink /etc/systemd/system/multi-user.target.wants/freeradius.service → /usr/lib/systemd/system/freeradius.service.
Configurando freeradius-utils (3.2.5+dfsg-3~ubuntu24.04.3) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Procesando disparadores para libc-bin (2.39-0ubuntu8.7) ...
ubuntu@rafae:~$
```

Puedes confirmar su instalación con el comando: `sudo systemctl status freeradius`
Esto indica que FreeRADIUS ya está operativo, pero todavía no hace nada útil porque no lo hemos configurado.

```
buntu@rafae:~$ sudo systemctl status freeradius
● freeradius.service - FreeRADIUS multi-protocol policy server
   Loaded: loaded (/usr/lib/systemd/system/freeradius.service; enabled; pres
   Active: active (running) since Wed 2026-04-15 11:43:37 -04; 5min ago
     Docs: man:radiusd(8)
           man:radiusd.conf(5)
           http://wiki.freeradius.org/
           http://networkradius.com/doc/
   Main PID: 13212 (freeradius)
    Status: "Processing requests"
     Tasks: 6 (limit: 4592)
  Memory: 43.4M (max: 2.0G available: 1.9G peak: 43.8M)
       CPU: 190ms
    CGroup: /system.slice/freeradius.service
            └─13212 /usr/sbin/freeradius -f
```

Para crear un usuario en FreeRadius, ingresas al siguiente comando en Ubuntu:

`sudo nano /etc/freeradius/3.0/users` que permitirá acceder a la configuración de este protocolo. Al final del archivo ingresa la siguiente línea:

adminradius Cleartext-Password := "123456"

```
# On no match, the user is denied access.

#####
# You should add test accounts to the TOP of this file! #
# See the example user "bob" above.                      #
#####

adminradius Cleartext-Password: = "12345"
Guardar el búfer modificado?
S Sí
N No      ^C Cancelar
```

Guarda el archivo

Existen 2 formas de levantar RADIUS: modo normal y modo debug. Cuando haces la instalación inicial de freeradius el servicio se levanta automáticamente en modo normal utilizando los puertos 1812, 1813, 18120, etc.

Esto es importante señalar, ya que, si quieres levantar el servicio en modo debug, deberás cerrar todos los procesos (incluyendo en segundo plano) de freeradius ya que no permite hacer el levantamiento dos veces en un mismo puerto.

El modo Debug nos ayuda a tener un diagnostico en caso de haber algún error en la configuración. Esto nos permite saber que es lo que está sucediendo y como se comporta Freeradius, ya que, al momento de levantar el servicio modo normal y este falla, el servicio no dará ningún indicador del fallo.

Recomendación: si estás en laboratorio para aprendizaje, utiliza el modo debug, pero si estás configurando en un entorno empresarial, utiliza el modo normal que ya se encontrará activo por defecto desde la instalación.

Para cerrar todos los procesos de Freeradius, debes ingresar los siguientes comandos:

- `sudo pkill freeradius`
- `sudo pkill radiusd`
- `ps aux | grep radius`

posteriormente levantas el servicio con: `sudo /usr/sbin/freeradius -X`

y en otra terminal ejecutas: `radtest adminradius 123456 127.0.0.1 0 testing123`

Una vez ejecutado, es importante revisar que al final del mensaje indique: **ready to process request**

Posteriormente abre una nueva ventana en Ubuntu y ejecuta: `radtest adminradius 123456 127.0.0.1 0 testing123`

```
ubuntu@rafae:~$ radtest adminradius 123456 127.0.0.1 0 testing123
Sent Access-Request Id 48 from 0.0.0.0:35154 to 127.0.0.1:1812 length 81
  User-Name = "adminradius"
  User-Password = "123456"
  NAS-IP-Address = 127.0.1.1
  NAS-Port = 0
  Message-Authenticator = 0x00
  Cleartext-Password = "123456"
Received Access-Accept Id 48 from 127.0.0.1:1812 to 127.0.0.1:35154 length 38
  Message-Authenticator = 0x52822ab278961375e1c953c5d78407ce
```

El resultado debe *indicar received Access Accept* para confirmar que el servidor está funcionando.

En este punto el objetivo logrado fue:

- levantar el servidor RADIUS
- definiste al usuario
- validaste autenticación real.

Vinculación de Ubuntu al dominio YORHA CENTER

A continuación, se hará que otra máquina (Windows server 2025) le pregunte a RADIUS.

El primer paso es asegurar de que Ubuntu esté direccionando el DNS de Windows server. Ingresas el comando para confirmar que sea la IP de Windows:
cat /etc/resolv.conf

En caso de configurar, ingresas a *sudo nano /etc/systemd/resolved.conf*, ingresas la IP en el DNS y el nombre del dominio en Domains como se visualiza en la imagen.

```
GNU nano 7.2 /etc/systemd/resolved.conf *
[Resolve]
# Some examples of DNS servers which may be used for DNS= and FallbackDNS=:
# Cloudflare: 1.1.1.1#cloudflare-dns.com 1.0.0.1#cloudflare-dns.com 2606:4700:4>
# Google:      8.8.8.8#dns.google 8.8.4.4#dns.google 2001:4860:4860::8888#dns.go>
# Quad9:       9.9.9.9#dns.quad9.net 149.112.112.112#dns.quad9.net 2620:fe::fe#d>
DNS=192.168.1.89
#FallbackDNS=
Domains=~yorha.center
#DNSSEC=no
#DNSOverTLS=no
#MulticastDNS=no
#LLMNR=no
#Cache=no-negative
#CacheFromLocalhost=no
#DNSStubListener=yes
#DNSStubListenerExtra=
#ReadEtcHosts=yes
#ResolveUnicastSingleLabel=no
#StaleRetentionSec=0

^G Ayuda      ^O Guardar    ^W Buscar     ^K Cortar     ^T Ejecutar   ^C Ubicación
^X Salir      ^R Leer fich. ^\ Reemplazar  ^U Pegar      ^J Justificar ^/ Ir a línea
```

Posteriormente reinicias con el comando: *sudo systemctl restart systemd-resolved* Y confirmas que esté bien configurado utilizando el comando: *resolvectl status*

```
rg@rafael:~$ resolvectl status
Global
    Protocols: -LLMNR -mDNS -DNSoverTLS DNSSEC=no/unsupported
    resolv.conf mode: stub
Current DNS Server: 192.168.1.89
    DNS Servers: 192.168.1.89
    DNS Domain: ~yorha.center

Link 2 (enp0s3)
    Current Scopes: DNS
    Protocols: +DefaultRoute -LLMNR -mDNS -DNSoverTLS DNSSEC=no/unsupported
    DNS Servers: 192.168.1.89
    DNS Domain: ~yorha.center
rg@rafael:~$
```

Último paso es probar el descubrimiento del DNS utilizando:

realm discover yorha.center

```
rg@rafael:~$ realm discover yorha.center
yorha.center
  type: kerberos
  realm-name: YORHA.CENTER
  domain-name: yorha.center
  configured: no
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
rg@rafael:~$
```

Con esta configuración, confirmas que Ubuntu ve y reconoce el dominio, Kerberos responde, DNS y AD están siendo reconocidos.

IMPORTANTE: INTERNAL UNEXPECTED ERROR JOINING THE DOMAIN

antes de unir Ubuntu al dominio revisar si está sincronizada el horario o de lo contrario no podrá unirse al dominio y presentará un el error interno de dominio.

se verifica la sincronización de hora con el comando: `timedatectl` el resultado en pantalla debe indicar *'sincronizado'*

si no está sincronizado debes ingresar el comando para regularizarlo

- `sudo timedatectl set-ntp true`
- `sudo systemctl restart systemd-timesyncd`

otro error común de esta vinculación es la configuración de Kerberos, ya sea por mal formato. En este caso debes ingresar lo siguiente:

```
sudo tee /etc/krb5.conf > /dev/null <<EOF
[libdefaults]
default_realm = YORHA.CENTER
dns_lookup_realm = false
dns_lookup_kdc = true
[realms]
YORHA.CENTER = {
kdc = 192.168.1.89
admin_server = 192.168.1.89
}
[domain_realm]
.yorha.center = YORHA.CENTER
yorha.center = YORHA.CENTER
EOF
```

Con esto limpiamos su configuración y utilizamos la IP directa. Posteriormente confirmar con el comando: `kinit Administrador@YORHA.CENTER` y luego: `klist`

si no hay problemas con este tipo de error puedes saltar este punto

Entonces ahora es necesario unir Ubuntu al dominio con el siguiente comando: *sudo realm join yorha.center -U Administrator*

Donde deberás ingresar la cuenta de usuario del administrador del dominio de Windows server:

```
rg@rafael:~$ sudo realm join yorha.center -U Administrator
[sudo] contraseña para rg:
Contraseña para Administrator:
rg@rafael:~$ realm list
yorha.center
  type: kerberos
  realm-name: YORHA.CENTER
  domain-name: yorha.center
  configured: kerberos-member
  server-software: active-directory
  client-software: sssd
  required-package: sssd-tools
  required-package: sssd
  required-package: libnss-sss
  required-package: libpam-sss
  required-package: adcli
  required-package: samba-common-bin
  login-formats: %U@yorha.center
  login-policy: allow-realm-logins
rg@rafael:~$
```

Esta pantalla te indica que Ubuntu se encuentra vinculado al dominio Yorha Center que está ubicado en Windows server 2025.

En este punto lograste:

- Linux unido a Active Directory
- Kerberos funcionando
- SSSD integrado
- Autenticación centralizada disponible

Vincular RADIUS con Active Directory

El primer paso es instalar winbind + herramientas de autenticación. En Ubuntu debes ejecutar lo siguiente: *sudo apt install samba winbind libpam-winbind libnss-winbind -y*

- Samba es implementar protocolos de Windows (SMB,NTLM) que es la base esencial para integrarse con A.D
- winbind que permite conectar Linux al dominio AD, esto permite consultar usuarios/grupos de dominio
- Libpam-winbind permite que usuarios de A.D puedan autenticarse en Linux

Con esto genera el flujo: usuario / RADIUS / ntlm_auth / winbind / Active Directory

Primero es necesario confirmar la configuración de los servicios instalados:

SMB: accede a *sudo nano /etc/samba/smb.conf* y debes ingresar el siguiente script para su correcta configuración:

```
[global]
workgroup = YORHA
security = ADS
realm = YORHA.CENTER
winbind use default domain = yes
winbind offline logon = no
winbind enum users = yes
winbind enum groups = yes
idmap config * : backend = tdb
idmap config * : range = 3000-7999
idmap config YORHA : backend = rid
idmap config YORHA : range = 10000-999999
template shell = /bin/bash
```

luego confirmar las sintaxis con el comando: *tesparm* y el resultado en pantalla debe indicar: *loaded services file OK*

reinicias el servicio:

```
sudo systemctl restart smbd
sudo systemctl restart winbind
```


IMPORTANTE: NT_STATUS_NO_LOGON_SERVERS

EN ALGUNAS CIRCUNSTANCIAS PUEDE QUE UBUNTU SI RECONOZCA EL DOMINIO, PERO NO SMB

Si este quedó mal configurado y SMB no reconoce el dominio debes seguir estos pasos:

1- Salir del dominio: `sudo realm leave yorha.center -U Administrador`

2- Confirma la salida de dominio con: `realm list`

3- Eliminar residuos guardados con el siguiente comando:

```
sudo systemctl stop winbind smbd
sudo rm -f /etc/krb5.keytab
sudo rm -f /var/lib/samba/*.tdb
sudo rm -f /var/lib/samba/private/*.tdb
```

4- Pega el siguiente script en la terminal:

```
sudo tee /etc/samba/smb.conf > /dev/null <<'EOF'
[global]
    workgroup = YORHA
    security = ADS
    realm = YORHA.CENTER
    server role = member server
    winbind use default domain = yes
    winbind offline logon = no
    winbind enum users = yes
    winbind enum groups = yes
    idmap config *: backend = tdb
    idmap config *: range = 3000-7999
    idmap config YORHA : backend = rid
    idmap config YORHA : range = 10000-999999
    template shell = /bin/bash
EOF
```

5- Unirse al dominio: `sudo net ads join -U Administrador`

6- Verificar que SMB reconozca el dominio: `sudo net ads testjoin`

SI NO PRESENTAS ESTE INCONVENIENTE PUEDES SALTARTE ESTA FASE

Finalizas levantando el servicio winbind con el comando: `sudo systemctl start winbind`
Y confirmas si está activo con el comando: `sudo systemctl status winbind --no-pager`

```
rg@rafael:~$ sudo systemctl start winbind
rg@rafael:~$ sudo systemctl status winbind --no-pager
● winbind.service - Samba Winbind Daemon
   Loaded: loaded (/usr/lib/systemd/system/winbind.service; disabled; preset: enabled)
   Active: active (running) since Wed 2026-04-15 20:33:24 -04; 6s ago
     Docs: man:winbindd(8)
           man:samba(7)
           man:smb.conf(5)
  Process: 10731 ExecCondition=/usr/share/samba/is-configured winbind (code=exited, status=0/SUCCESS)
 Main PID: 10734 (winbindd)
    Status: "winbindd: ready to serve connections..."
     Tasks: 5 (limit: 5588)
  Memory: 18.5M (peak: 34.8M)
     CPU: 365ms
    CGroup: /system.slice/winbind.service
            └─10734 /usr/sbin/winbindd --foreground --no-process-group
              └─10737 "winbindd: domain child [RAFAEL]"
                └─10738 "winbindd: domain child [YORHA]"
                  └─10740 /usr/libexec/samba/samba-dcerpcd --libexec-rpcds --ready-s...
                    └─10749 /usr/libexec/samba/rpcd_lsad --configfile=/etc/samba/smb.c...

abr 15 20:33:24 rafael winbindd[10734]: [2026/04/15 20:33:24.605047, 0] so...ain)
abr 15 20:33:24 rafael winbindd[10734]: winbindd version 4.19.5-Ubuntu st...ted.
abr 15 20:33:24 rafael winbindd[10734]: Copyright Andrew Tridgett and the...2023
abr 15 20:33:24 rafael systemd[1]: Started winbind.service - Samba Winbind ...mon.
abr 15 20:33:24 rafael samba-dcerpcd[10740]: [2026/04/15 20:33:24.634497, 0...in)
```

Luego valida si Ubuntu puede ver los usuarios y/o grupos de Active directory con el comando: `wbinfo -u` y `wbinfo -g`

```
rg@rafael:~$ wbinfo -u
administrador
invitado
krbtgt
fguajardo
msol_27f2d9d5b637
rg@rafael:~$ wbinfo -g
equipos del dominio
controladores de dominio
administradores de esquema
administradores de empresas
publicadores de certificados
admins. del dominio
usuarios del dominio
invitados del dominio
propietarios del creador de directivas de grupo
servidores ras e ias
grupo de replicación de contraseña rodc permitida
grupo de replicación de contraseña rodc denegada
controladores de dominio de sólo lectura
enterprise domain controllers de sólo lectura
controladores de dominio clonables
protected users
administradores clave
administradores clave de la organización
cuentas de confianza de bosque
cuentas de confianza externas
dnsadmins
dnsupdateproxy
adsyncadmins
adsyncoperators
adsyncbrowse
adsyncpasswordset
rg@rafael:~$
```

Como puedes ver, Ubuntu reconoce a los usuarios y grupos de Active Directory. Ingresa la siguiente línea de comando que permitirá validar credenciales contra AD, Windbind funciona y RADIUS ya puede utilizar A.D:

```
ntlm_auth --request-nt-key --domain=YORHA --username=Administrador
```

```
rg@rafael:~$ ntlm_auth --request-nt-key --domain=YORHA --username=Administrador
Password:
: (0x0)
rg@rafael:~$
```

Con esto confirmas que si está funcionando. (0X0) = OK

El siguiente paso es configurar el módulo MSCHAP en FreeRADIUS: ingresas al archivo:

```
sudo nano /etc/freeradius/3.0/mods-enabled/mschap
```

Debes buscar la línea que indica: `# ntlm_auth = "/path/to/ntlm_auth ..."`

```
nano 7.2 /etc/freeradius/3.0/mods-enabled/mschap
#
# ntlm_auth = mschapv2-and-ntlmv2-only
#
# This will let Samba 4 accept the MS-CHAP authentication
# method that is needed by FreeRADIUS.
#
# Depending on the Samba version, you may also need to add:
#
# --allow-mschapv2
#
# to the command-line parameters.
#
ntlm_auth = "/path/to/ntlm_auth --request-nt-key --allow-mschapv2 --username=%{Stripped-User-Name}:-%{User-Name}:-None} --challenge=%{mschap:u
#
# The default is to wait 10 seconds for ntlm_auth to
# complete. This is a long time, and if it's taking that
# long then you likely have other problems in your domain.
# The length of time can be decreased with the following
# option, which can save clients waiting if your ntlm_auth
```

y remplazarla por: `ntlm_auth = "/usr/bin/ntlm_auth --request-nt-key --domain=YORHA --username=%{User-Name} --password=%{User-Password}"`

```
#
# ntlm_auth = mschapv2-and-ntlmv2-only
#
# This will let Samba 4 accept the MS-CHAP authentication
# method that is needed by FreeRADIUS.
#
# Depending on the Samba version, you may also need to add:
#
# --allow-mschapv2
#
# to the command-line parameters.
#
ntlm_auth = "/usr/bin/ntlm_auth --request-nt-key --domain=YORHA --username=%{mschap:User-Name} --password=%{User-Password}"
#
# The default is to wait 10 seconds for ntlm_auth to
```

Y luego reinicias FreeRadius en modo Debug: `sudo systemctl stop freeradius` Luego levanta el debug: `sudo freeradius -X`

En otra terminal ingresas la siguiente línea de comando para ejecutar el servicio: `radtest Administrador 123456 127.0.0.1 0 testing123`

IMPORTANTE: [mschap] = noop pap: WARNING: No "known good" password found for the user ERROR: No Auth-Type found: rejecting the user

Este error se debe a que el servidor recibió tu usuario y contraseña, pero no supo qué hacer con ellos. Primero intentó usar el método que conecta con Active Directory (MSCHAP), pero no se activó. Después probó con el método simple (PAP), pero como ya no tiene contraseñas guardadas localmente, tampoco pudo validar. Entonces quedó en una situación donde tiene los datos, pero no tiene una forma definida de verificarlos, y por eso decide rechazar el acceso.

PAP: es el método más simple: el usuario envía directamente su usuario y contraseña, y el servidor la compara. Es fácil de usar, pero menos seguro porque la contraseña viaja “tal cual”.

MSCHAP: en cambio, es más seguro: el usuario no envía la contraseña directamente, sino una prueba cifrada (un “desafío y respuesta”) que demuestra que la conoce. Es el método que usa Windows y Active Directory para autenticación.

En resumen: PAP envía la contraseña, mientras que MSCHAP demuestra que la sabes sin enviarla.

¿Cómo regularizar este inconveniente?

Tienes dos formas de configurar la autenticación:

- La segunda es usar clientes reales como WiFi o VPN, que trabajan con MSCHAPv2 y representan un entorno empresarial más cercano a la realidad.
- La tercera es utilizar herramientas como radtest con MSCHAP directamente, lo que es más correcto a nivel técnico, pero también más complejo de implementar.

Ingresa en el terminal el siguiente comando: *sudo nano /etc/freeradius/3.0/sites-enabled/default*

Y en el bloque de `authorize` donde indica `files` ingresa lo siguiente:

```
if (&User-Password) {  
  update control {  
    Auth-Type := mschap  
  }  
}
```

En producción debe quedar de la siguiente configuración. Guardas el archivo y reinicias RADIUS

```
# Unix

#
# Read the 'users' file. In v3, this is located in
# raddb/mods-config/files/authorize
files
expiration
logintime
pap

if (&User-Password) {
    update control {
        Auth-Type := mschap
    }
}

#
# Look in an SQL database. The schema of the database
# is meant to mirror the "users" file.
#
# See "Authorization Queries" in mods-available/sql
-sql
```

Reinicias RADIUS:

```
sudo systemctl stop freeradius
```

```
sudo freeradius -X
```

una vez reiniciado confirmas con el comando: `radtest "YORHA\\Administrador" Yorha11945 127.0.0.1 0 testing123`

```
rg@rafael:~$ sudo nano /etc/freeradius/3.0/sites-enabled/default
rg@rafael:~$ radtest "YORHA\\Administrador" Yorha11945 127.0.0.1 0 testing123
Sent Access-Request Id 112 from 0.0.0.0:60912 to 127.0.0.1:1812 length 89
    User-Name = "YORHA\\Administrador"
    User-Password = "Yorha11945"
    NAS-IP-Address = 127.0.1.1
    NAS-Port = 0
    Message-Authenticator = 0x00
    Cleartext-Password = "Yorha11945"
Received Access-Reject Id 112 from 127.0.0.1:1812 to 127.0.0.1:60912 length 38
    Message-Authenticator = 0x4755309e3a99e72f2e1a42fcb0641e0c
(0) -: Expected Access-Accept got Access-Reject
```

Con esta configuración RADIUS puede autenticar contra Active Directory. Quedando en funcionamiento:

- ntlm_auth
- wbinfo
- real join
- kinit

no obstante, la imagen muestra que el servidor recibe correctamente el usuario y la contraseña, pero responde con **Access-Reject** porque está usando radtest, que envía autenticación simple (PAP), mientras que tu configuración con Active Directory funciona con MSCHAP; por eso, aunque todo el sistema está bien integrado, el método de prueba no es compatible y la autenticación es rechazada.

Conclusión:

La implementación de FreeRADIUS integrado con Active Directory representa una mejora significativa en la gestión de identidades y control de accesos dentro de un entorno organizacional. Al centralizar la autenticación en el dominio, se eliminan credenciales locales dispersas, reduciendo la superficie de ataque y evitando inconsistencias en la administración de usuarios. Además, permite aplicar políticas de seguridad de forma uniforme, como complejidad de contraseñas, bloqueo de cuentas y control de accesos basado en roles.

Desde el punto de vista de ciberseguridad, esta arquitectura entrega mayor trazabilidad y visibilidad, ya que todos los intentos de autenticación pueden ser monitoreados y correlacionados, facilitando la detección de actividades sospechosas como intentos de fuerza bruta o accesos no autorizados. La integración con protocolos más seguros como MSCHAPv2, y la posibilidad de escalar hacia mecanismos más robustos como autenticación multifactor (MFA) o certificados digitales, permite fortalecer aún más la protección de los accesos.

En conjunto, este tipo de implementación no solo mejora la seguridad técnica, sino que también alinea la infraestructura con buenas prácticas y estándares utilizados en entornos empresariales, aportando control, escalabilidad y una base sólida para futuras estrategias de defensa en profundidad.